Linear Velocity Calibration

Linear Velocity Calibration

1. Course Content
2. Preparation
3. Calibration Demonstration
4. Source Code Explanation

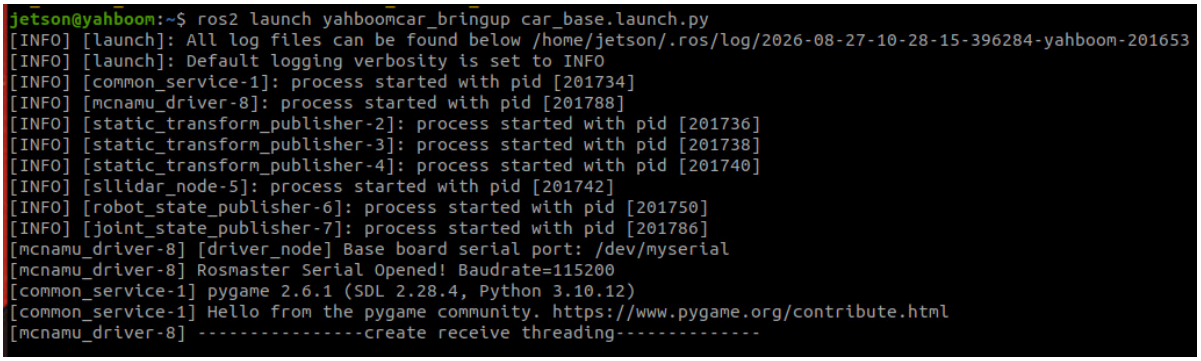
1. Course Content

- Use a reference object of known length, such as a tape measure, to verify the accuracy of the chassis wheel odometry (computed from motor encoder data) and perform linear velocity calibration

> **Important**
>
> The car has been calibrated at the factory; there is no need to calibrate it again on first use. If you find that the odometry error is relatively large during subsequent use, refer to this tutorial to calibrate

2. Preparation

```
ros2 launch yahboomcar_bringup car_base.launch.py
```



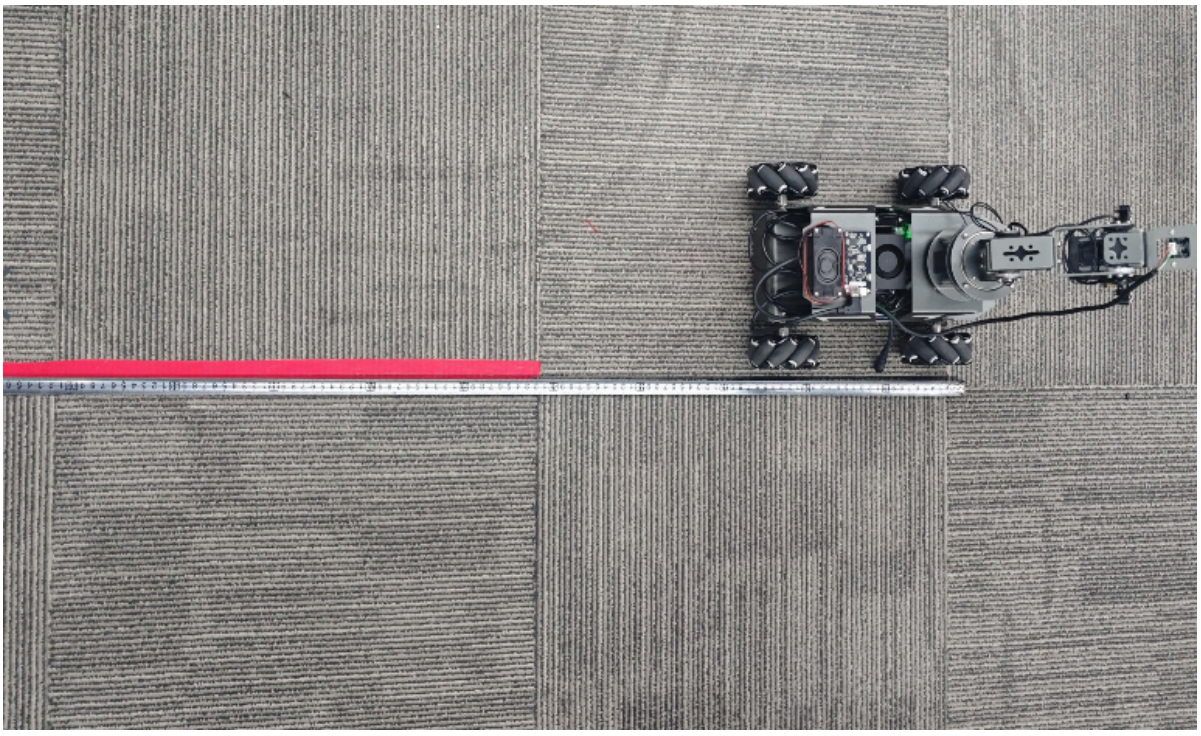
3. Calibration Demonstration

- The demonstration uses the front of the car as the motion distance reference. Run the linear velocity calibration program

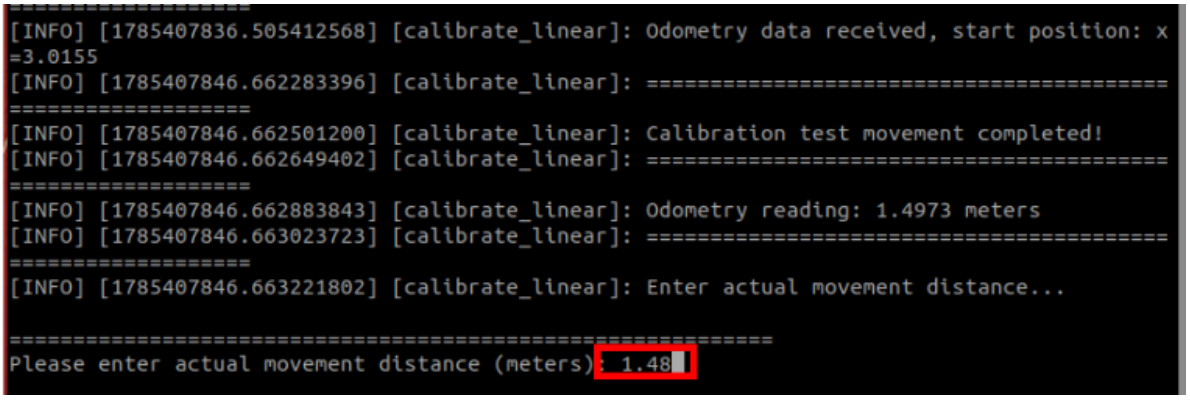
```
ros2 run common_demo calibrate_linear
```

> **Tip**
>
> Optional additional startup parameters

Parameter Name	Type	Default Value	Description
test_distance	float	1.5	Test movement distance (meters)
speed	float	0.2	Movement speed (m/s)
tolerance	float	0.01	Allowed error (meters)
direction	str	x	Test linear velocity direction, values x, y



- After the chassis finishes moving, use a tape measure or a standard scale reference to measure the chassis's actual movement distance, and enter the chassis's actual movement distance

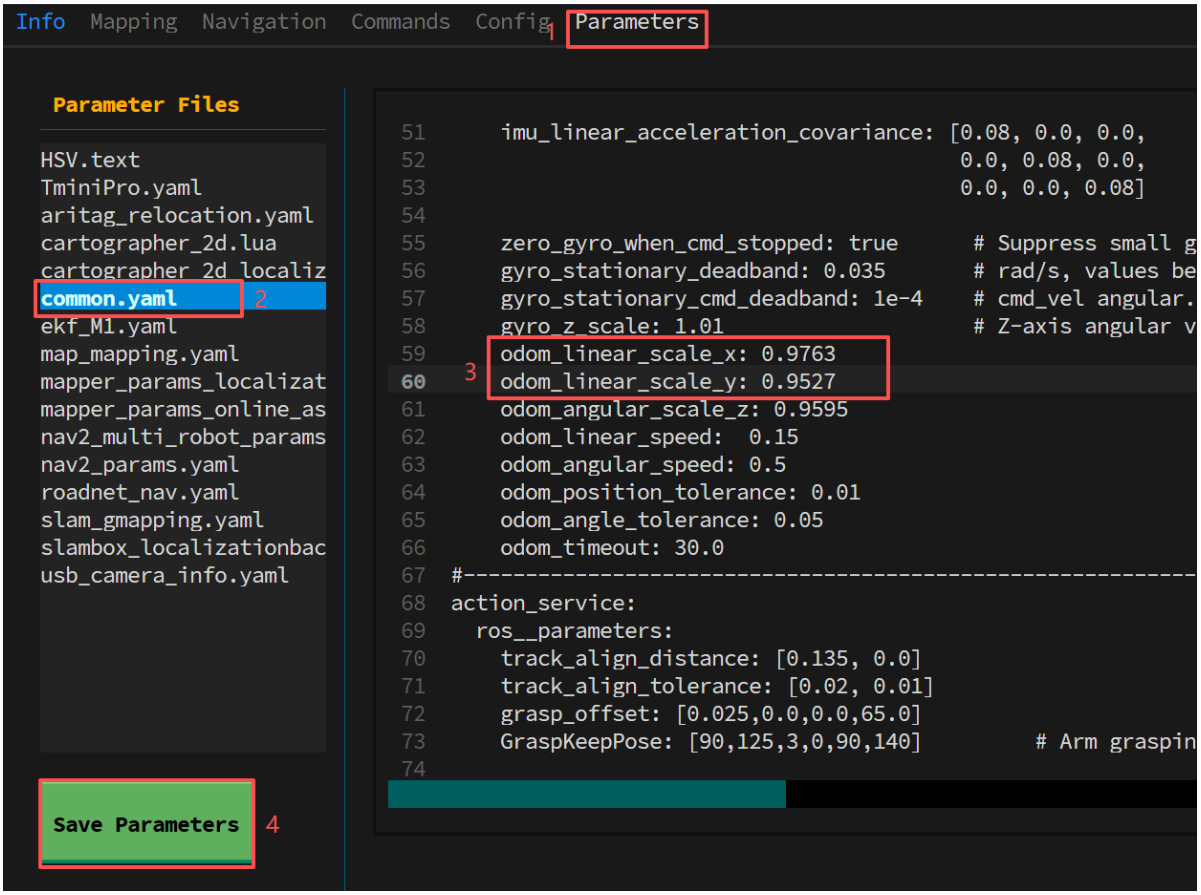


- Repeat the above calibration process to calibrate the y direction

```
ros2 run common_demo calibrate_linear --ros-args -p direction:='"y"'
```

- Apply the calibration coefficient

```
robot_manage gui
```



4. Source Code Explanation

- Source code path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/calibration/ca
librate_linear.py
```

1. `odom_callback` — Odometry data callback

- Extract the current `x` and `y` coordinates from the odometry message
- On the first reception, record the starting position `start_x / start_y` and set the `odom_received` flag

2. `control_loop` — Calibration control main loop

- Calculate the distance moved along the x-axis `moved_distance` and the target error `error`
- When the error is within the tolerance range, stop moving, output the odometry reading, and set `waiting_for_input` to wait for the user to enter the actual distance
- When the target has not been reached, decelerate in stages based on the remaining distance (< 0.1m minimum speed, 0.1~0.3m linear deceleration, > 0.3m full speed)
- Use `copysign` to determine the velocity direction and publish a `Twist` velocity command

```
    def odom_callback(self, msg):
        """Odometry data callback"""
        self.current_x = msg.pose.pose.position.x
        self.current_y = msg.pose.pose.position.y

        if not self.odom_received:
            self.odom_received = True
            self.start_x = self.current_x
            self.start_y = self.current_y
            self.get_logger().info(f'Odometry data received, start position: x=
{self.start_x:.4f}, y={self.start_y:.4f}')

    def control_loop(self):
        if not self.odom_received:
            return

        if not self.test_started:
            self.start_x = self.current_x
            self.start_y = self.current_y
            self.test_started = True

        # Calculate moved distance (X-axis only)
        moved_distance = self.current_x - self.start_x

        # Calculate error
        error = self.test_distance - abs(moved_distance)

        # Check if target reached
        if abs(error) < self.tolerance and not self.test_completed:
            self.test_completed = True
            self.get_logger().info('='*60)
            self.get_logger().info('Calibration test movement completed!')
            self.get_logger().info('='*60)
            # Stop robot
            self.cmd_vel_pub.publish(Twist())
            self.timer.cancel()
            # Display odometry reading
            self.odom_distance = abs(moved_distance)
            self.get_logger().info(f'Odometry reading: {self.odom_distance:.4f}
meters')
            self.get_logger().info('='*60)
            self.get_logger().info('Enter actual movement distance...')

            # Set flag to notify main function to wait for user input
            self.waiting_for_input = True
            return

        # Target not reached, continue moving
        move_cmd = Twist()
        distance_to_go = abs(error)

        # Velocity ramping: reduce speed when approaching target to prevent
overshoot
        min_speed = max(0.03, self.speed * 0.2)
        close_threshold = 0.1    # Below this: minimum speed
        ramp_threshold = 0.3     # Below this: linear deceleration
```

```
        if distance_to_go < close_threshold:
            ramp_speed = min_speed
        elif distance_to_go < ramp_threshold:
            ramp_speed = min_speed + (self.speed - min_speed) * (
                (distance_to_go - close_threshold) / (ramp_threshold -
close_threshold))
        else:
            ramp_speed = self.speed

        move_cmd.linear.x = copysign(ramp_speed, error)
        self.cmd_vel_pub.publish(move_cmd)
```